SQL FOR DATA ANALYSIS · LESSON 17 OF 20

Percentiles, Medians and Distribution Queries

Go beyond averages. Compute medians and percentiles, split customers into quartiles and deciles, measure how skewed a distribution is, and find outliers with SQL that works on real retail data.

Gap-fill lesson | 2026 | Advanced | 120 min

By the end of this lesson you can

- Explain why the median and percentiles can be more honest than the mean for skewed money data
- Compute medians and arbitrary percentiles with PERCENTILE_CONT and PERCENTILE_DISC
- Bucket rows into quartiles and deciles with NTILE and percent-rank measures
- Flag outliers with the interquartile range

Datasets used

`superstore_sales.csv`

Why averages mislead

Order values in retail are skewed: most orders are small, and a few very large ones pull the average up. The mean order value may look healthy while the typical customer spends much less. The **median** (the middle value) and **percentiles** (values below which a chosen share of rows fall) describe the typical case and the spread.

CASE STUDY **Is the average order really typical?**

The sales director quotes "average order value of about 2,000 rupees". The question is whether that describes a typical order, or whether a few big corporate orders dominate it. Answering needs the median, the quartiles and the top percentile, not one average.

Median and percentiles with PERCENTILE_CONT

`PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)` returns the value at fraction `p`, interpolating between rows when needed. `p = 0.5` is the median.

SQL

```sql
WITH order_totals AS (
    SELECT order_id, SUM(sales) AS order_value
    FROM superstore
    GROUP BY order_id
)
SELECT
    ROUND(AVG(order_value), 0)                                          AS
mean_order_value,
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY order_value), 0)    AS p25,
    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY order_value), 0)    AS median,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY order_value), 0)    AS p75,
    ROUND(PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY order_value), 0)    AS p90,
    ROUND(MAX(order_value), 0)                                           AS
largest_order
FROM order_totals;
```

RESULT

mean_order_value	p25	median	p75	p90	largest_order
62,942	6,526	26,101	78,458	178,860	622,758

1 rows × 6 columns

If the mean is far above the median, a small share of large orders is pulling it up. Report both, and say which one describes a typical order.

PERCENTILE_CONT versus PERCENTILE_DISC: `PERCENTILE_CONT` interpolates between two values; `PERCENTILE_DISC` returns an actual value from the data. Use `PERCENTILE_DISC` when the answer must be a real observed order, such as "the order value that 90% of orders are below".

Quartiles and deciles with NTILE

`NTILE(n)` assigns each row to one of n equal-sized groups. It is the easiest way to segment customers by spend. The query below ranks customers into quartiles by lifetime sales.

SQL

```sql
WITH customer_sales AS (
    SELECT customer_id, SUM(sales) AS lifetime_sales
    FROM superstore
    GROUP BY customer_id
),
quartiles AS (
    SELECT customer_id, lifetime_sales,
           NTILE(4) OVER (ORDER BY lifetime_sales) AS spend_quartile
    FROM customer_sales
)
SELECT spend_quartile,
       COUNT(*)                          AS customers,
       ROUND(MIN(lifetime_sales), 0)     AS min_spend,
       ROUND(MAX(lifetime_sales), 0)     AS max_spend,
       ROUND(SUM(lifetime_sales), 0)     AS total_spend,
       ROUND(100.0 * SUM(lifetime_sales) / SUM(SUM(lifetime_sales)) OVER (), 1) AS
share_of_sales_pct
FROM quartiles
GROUP BY spend_quartile
ORDER BY spend_quartile;
```

RESULT

spend_quartile	customers	min_spend	max_spend	total_spend	share_of_sales_pct
1	197	140	88,830	7,935,534	3.9
2	196	89,102	200,899	28,192,185	14.0
3	196	201,378	373,460	54,558,902	27.1
4	196	373,864	1,204,439	110,726,949	55.0

4 rows × 6 columns

Quartile 4 is the top 25% of customers by spend. The share column usually shows that a small quartile produces a large share of revenue. That is the Pareto pattern that marketing budgets should reflect.

::::note Ties and buckets `NTILE` splits rows by position, so customers with identical spend can land in different buckets. If you need every customer with the same value to share one bucket, use `PERCENT_RANK()` or `CUME_DIST()` and cut at thresholds instead. ::::

Percent rank and cumulative distribution

`PERCENT_RANK()` gives each row a value from 0 to 1 showing its relative position. `CUME_DIST()` gives the share of rows at or below its value. Both answer "what share of customers spend less than

this one?"

```sql
WITH customer_sales AS (
    SELECT customer_id, SUM(sales) AS lifetime_sales
    FROM superstore
    GROUP BY customer_id
)
SELECT customer_id,
       ROUND(lifetime_sales, 0) AS lifetime_sales,
       ROUND(PERCENT_RANK() OVER (ORDER BY lifetime_sales), 3) AS percent_rank,
       ROUND(CUME_DIST()    OVER (ORDER BY lifetime_sales), 3) AS cume_dist
FROM customer_sales
ORDER BY lifetime_sales DESC
LIMIT 10;
```

RESULT

customer_id	lifetime_sales	percent_rank	cume_dist
C0741	1,204,439	1.00	1.00
C0092	1,140,540	1.00	1.00
C0306	1,126,867	1.00	1.00
C0781	1,123,527	1.00	1.00
C0443	1,113,058	0.99	0.99
C0720	1,100,213	0.99	0.99
C0493	1,004,382	0.99	0.99
C0028	1,000,189	0.99	0.99
C0630	1,000,175	0.99	0.99
C0262	952,738	0.99	0.99

10 rows × 4 columns

Use these measures when you want to say "this customer is in the top 5%" without forcing exact bucket sizes.

Spread: standard deviation and the interquartile range

The median says where the middle is; the spread says how much values vary. Two measures are common:

- **Standard deviation** (STDDEV_SAMP) measures typical distance from the mean. It assumes a roughly symmetric shape.
- **Interquartile range** (IQR = P75 minus P25) measures the spread of the middle half. It is robust to outliers.

The next query computes the IQR for profit per line and counts lines outside the usual outlier fences (1.5 times the IQR beyond the quartiles).

SQL

```
WITH q AS (
    SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY profit) AS q1,
           PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY profit) AS q3
    FROM order_items
),
fences AS (
    SELECT q1, q3, q3 - q1 AS iqr,
           q1 - 1.5 * (q3 - q1) AS lower_fence,
           q3 + 1.5 * (q3 - q1) AS upper_fence
    FROM q
)
SELECT ROUND(f.q1, 1) AS q1, ROUND(f.q3, 1) AS q3, ROUND(f.iqr, 1) AS iqr,
       ROUND(f.lower_fence, 1) AS lower_fence, ROUND(f.upper_fence, 1) AS upper_fence,
       COUNT(*) FILTER (WHERE i.profit < f.lower_fence OR i.profit > f.upper_fence) AS
outlier_lines,
       COUNT(*) AS all_lines
FROM order_items i
CROSS JOIN fences f
GROUP BY f.q1, f.q3, f.iqr, f.lower_fence, f.upper_fence;
```

RESULT

q1	q3	iqr	lower_fence	upper_fence	outlier_lines	all_lines
-17.9	2,270.1	2,288.1	-3,450.1	5,702.3	1361	6323

1 rows × 7 columns

Outliers are not automatically errors. A very large profitable line may be a genuine bulk deal. Investigate each one before excluding it.

Skewness check in plain SQL

A quick comparison of mean and median shows the direction of skew. If the mean is greater than the median, the distribution has a long right tail; if it is smaller, a long left tail. The query below reports that gap for sales per line by category.

SQL

```sql
SELECT p.category,
       COUNT(*)                                                    AS lines,
       ROUND(AVG(i.sales), 0)                                      AS mean_sales,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY i.sales), 0)  AS median_sales,
       ROUND(AVG(i.sales) - PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY i.sales), 0) AS
mean_minus_median
FROM order_items i
JOIN products p ON p.product_id = i.product_id
GROUP BY p.category
ORDER BY mean_minus_median DESC;
```

RESULT

category	lines	mean_sales	median_sales	mean_minus_median
Technology	1892	82,684	56,789	25,895
Furniture	1917	18,175	12,970	5,205
Office Supplies	2514	4,031	1,152	2,879

3 rows × 5 columns

A large positive gap in a category means a few expensive lines dominate that category's average, which affects how you set targets for it.

::::warn Averages of averages Never average a set of averages (for example, averaging each region's mean order value) and call it the overall mean unless the groups are the same size. Recompute from the underlying rows, or use the median. ::::

::::key What to remember - Report the median and percentiles beside the mean for money data, which is usually skewed. - `PERCENTILE_CONT` interpolates; `PERCENTILE_DISC` returns an actual observed value. - `NTILE` gives equal-sized groups by position; `PERCENT_RANK` and `CUME_DIST` give relative positions. - The IQR and its fences flag outliers without being distorted by them. - Investigate outliers before you remove them. ::::

Quick quiz

5 questions · tap an answer to check it

1. A category has a mean sale of 5,000 and a median of 1,200. What does this suggest?

A Most lines are above average

B A few very large lines pull the mean up; the typical line is much smaller

C The data has no outliers

D The median is wrong

Answer: B. A mean far above the median signals a long right tail.

2. Which function returns the median?

A AVG(x)

B MODE(x) over the whole table

C PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)

D COUNT(DISTINCT x)

Answer: C. Fraction 0.5 is the 50th percentile, which is the median.

3. You need an actual observed order value as the 90th percentile. Which is better?

A PERCENTILE_CONT

B PERCENTILE_DISC

C AVG

D STDDEV_SAMP

Answer: B. PERCENTILE_DISC returns a value that exists in the data.

4. NTILE(4) over customers ordered by spend creates:

A Four customers

B Four groups of roughly equal size

C Four groups of equal spend

D The top four customers

Answer: B. NTILE splits rows by position into equal-sized buckets.

5. The interquartile range is calculated as:

A Maximum minus minimum

B Mean minus median

C P75 minus P25

D Standard deviation times 2

Answer: C. IQR spans the middle half of the data.

EXERCISE **Concentration of revenue**

Write one query that returns, for each product category, the median order line sales, the 90th percentile line sales, and the share of total sales coming from lines above the category's 90th percentile.

Show solution (worked answer)

Hint: compute the 90th percentile per category with `PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY sales)` in a CTE grouped by category, join it back to `order_items` through `products`, then use `SUM(CASE WHEN sales > p90 THEN sales END)` divided by the total sales for the category.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.